

FamilleConnect — Architecture Technique

Front-end, Back-end, API, Base de données, Hébergement

Document : Architecture technique appliquée de la plateforme

Version : 1.0

Date : Septembre 2026

Statut : Pour validation

Basé sur : Spécifications fonctionnelles des 7 univers

Audience : Équipe technique, CTO, comité de pilotage, partenaires techniques

Table des matières

Synthèse exécutive.....	1
Les chiffres clés.....	2
1. Vue d'ensemble de l'architecture.....	2
1.1 Principes directeurs.....	2
1.2 Schéma global de l'architecture.....	3
1.3 Flux principaux.....	10
2. Contraintes techniques du contexte camerounais.....	10
2.1 Connectivité intermittente.....	11
2.2 Mobile Money dominant.....	11
2.3 Multi-fuseaux diaspora.....	12
2.4 Coût et pouvoir d'achat.....	13
2.5 Données sensibles et réglementation.....	14
3. Front-end mobile.....	15
3.1 Choix du framework : React Native.....	15
3.2 Architecture offline-first.....	16
3.3 Gestion des notifications push.....	17
3.4 Accessibilité et multi-langue.....	18
3.5 Sécurité mobile.....	19
3.6 Distribution et mises à jour.....	19
4. Front-end web.....	20
4.1 Choix du framework : Next.js 16.....	20
4.2 Architecture web.....	21
4.3 Mode hors-ligne léger (PWA).....	22
4.4 Optimisation diaspora.....	23
4.5 Accessibilité web.....	24
5. Back-end.....	24

5.1 Choix du framework : NestJS.....	25
5.2 Architecture en microservices modérés.....	25
5.3 Workflow Engine.....	28
5.4 Notification Engine.....	29
5.5 Auth Service.....	30
6. API et interfaces.....	31
6.1 Trois types d'API.....	31
6.2 REST API.....	32
6.3 GraphQL API.....	33
6.4 WebSocket API.....	34
6.5 Webhooks.....	35
6.6 Rate limiting et sécurité API.....	36
7. Base de données.....	37
7.1 Vue d'ensemble.....	37
7.2 PostgreSQL — base principale.....	38
7.2.1 Schéma de données.....	38
7.2.2 Immutabilité.....	40
7.2.3 Réplication.....	40
7.3 Redis — cache et messages.....	41
7.4 Elasticsearch — recherche full-text.....	42
7.5 Stockage objet S3-compatible.....	43
7.6 Stratégie de sauvegarde.....	46
8. Hébergement et infrastructure.....	47
8.1 Choix du fournisseur cloud.....	47
8.2 Architecture d'hébergement.....	48
8.3 Composants d'infrastructure.....	49
8.4 Conteneurisation et orchestration.....	50
8.5 Plan de continuité d'activité.....	51
8.6 Hébergement local envisageable.....	51
9. Sécurité technique transverse.....	52

9.1 Authentification.....	52
9.2 Autorisation.....	53
9.3 Chiffrement.....	54
9.4 Conformité RGPD-like.....	54
9.5 Gestion des secrets.....	55
10. Performance et scalabilité.....	56
10.1 Stratégie de cache.....	56
10.2 Optimisation des requêtes.....	57
10.3 Scalabilité horizontale.....	58
10.4 Gestion des pics.....	58
10.5 Monitoring.....	59
11. Intégrations externes.....	60
11.1 MTN MoMo API.....	60
11.2 Orange Money API.....	61
11.3 Calendriers externes.....	61
11.4 SMS gateway.....	62
11.5 E-mail.....	62
11.6 Push notifications.....	63
11.7 Visioconférence (Phase 2).....	64
12. Stack technique récapitulative.....	64
13. Coûts d'infrastructure indicatifs.....	64
13.1 Phase 1 — MVP (5-10 familles, 1 500-3 000 membres).....	64
13.2 Phase 2 — Enrichissement (30-100 familles, 9 000-30 000 membres).....	65
13.3 Phase 3 — Maturité (500-2 000 familles, 150 000-600 000 membres).....	65
13.4 Coût par membre.....	65
14. Risques techniques et mitigations.....	66
15. Plan de déploiement et CI/CD.....	66
15.1 Environnements.....	66
15.2 CI/CD.....	67
15.3 Monitoring post-déploiement.....	67

16. Conclusion.....	68
Principes à retenir.....	68

Note : Cette table des matières est générée par champ Word. Pour actualiser les numéros de page après édition, faites un clic droit sur la table puis sélectionnez « Mettre à jour les champs ».

Synthèse exécutive

Ce document définit l'architecture technique appliquée de la plateforme FamilleConnect, couvrant les cinq piliers fondamentaux : front-end mobile, front-end web, back-end, API et base de données, hébergement. Il traduit les spécifications fonctionnelles des 7 univers en choix techniques concrets, adaptés aux réalités camerounaises et africaines, et fournit une feuille de route d'implémentation claire.

L'architecture proposée repose sur cinq principes directeurs qui répondent aux contraintes du contexte : offline-first (connectivité intermittente en zones rurales), security by design (données sensibles familiales), scalabilité maîtrisée (de 5 familles pilotes à plusieurs milliers), coût optimisé (modèle économique accessible), et interopérabilité native (Mobile Money, calendriers externes, multi-appareils). Ces principes guident tous les choix techniques, depuis le framework front-end jusqu'à la stratégie d'hébergement.

La stack technique recommandée privilégie la maturité, la communauté, et la maintenabilité : React Native pour le mobile (code partagé iOS/Android, écosystème riche), Next.js 16 pour le web (SSR/SSG, performance, SEO), NestJS pour le back-end (architecture modulaire, TypeScript natif), PostgreSQL pour la base principale (ACID, immutabilité, maturité), et un hébergement cloud multi-zones avec CDN pour la diaspora. Les coûts d'infrastructure sont estimés à 2-3 millions FCFA par mois en régime de croisière (1 000 familles), intégrables au modèle économique.

Les chiffres clés

5 piliers de l'architecture technique

Front-end mobile, front-end web, back-end, API + données, hébergement

5 principes directeurs

Offline-first, security by design, scalabilité, coût optimisé, interopérabilité

< 500 FCFA par membre et par mois (cible infrastructure)

Atteint dès la Phase 2, optimisé en Phase 3

13 modules back-end (7 univers + 6 transverses)

Architecture modulaire NestJS, prête pour microservices

1. Vue d'ensemble de l'architecture

1.1 Principes directeurs

L'architecture technique de FamilleConnect est guidée par cinq principes directeurs, qui découlent directement des contraintes du contexte camerounais et des exigences fonctionnelles des 7 univers.

- **Offline-first.** La connectivité Internet n'est pas garantie au Cameroun, notamment en zones rurales (villages d'origine, congrès en brousse). L'application doit fonctionner en mode hors-ligne pour les opérations critiques (consultation du registre, saisie d'un paiement en espèces, consultation des soldes) et synchroniser automatiquement au retour de la connexion. Ce principe influence tous les choix techniques : base de données locale sur mobile, synchronisation différée, gestion des conflits.
- **Security by design.** FamilleConnect gère des données sensibles : santé, précarité, conflits familiaux, testaments, finances. La sécurité n'est pas une couche ajoutée, mais un principe structurant : chiffrement au repos et en transit, authentification à deux facteurs, journalisation immuable, principe du moindre privilège, audits réguliers. Ce principe est non-négociable, car une faille pourrait discréditer définitivement la plateforme.
- **Scalabilité maîtrisée.** La plateforme doit passer de 5 familles pilotes (1 500 membres) à plusieurs milliers de familles (plusieurs centaines de milliers de membres) sans rupture architecturale. L'approche retenue est la scalabilité horizontale (microservices modérés, base de données avec réplica en lecture, cache distribué) plutôt que verticale (serveur plus gros). Cette approche permet une croissance progressive des coûts, alignée sur les revenus.
- **Coût optimisé.** Le modèle économique repose sur des abonnements familiaux modiques (10 000 à 25 000 FCFA/an). L'infrastructure ne peut pas être luxueuse. Les choix techniques privilégient les solutions open-source, les services managés économiques, et l'optimisation continue des coûts. Un membre coûte moins de 500 FCFA par mois en infrastructure à maturité.
- **Interopérabilité native.** FamilleConnect ne vit pas en isolation : elle s'intègre avec MTN MoMo, Orange Money, Google Calendar, Apple Calendar, Outlook, des services de visioconférence, des passerelles SMS, des services de push. Ces intégrations sont des

facteurs critiques d'adoption, et l'architecture les facilite par des interfaces claires, des webhooks, et des adaptateurs dédiés.

1.2 Schéma global de l'architecture

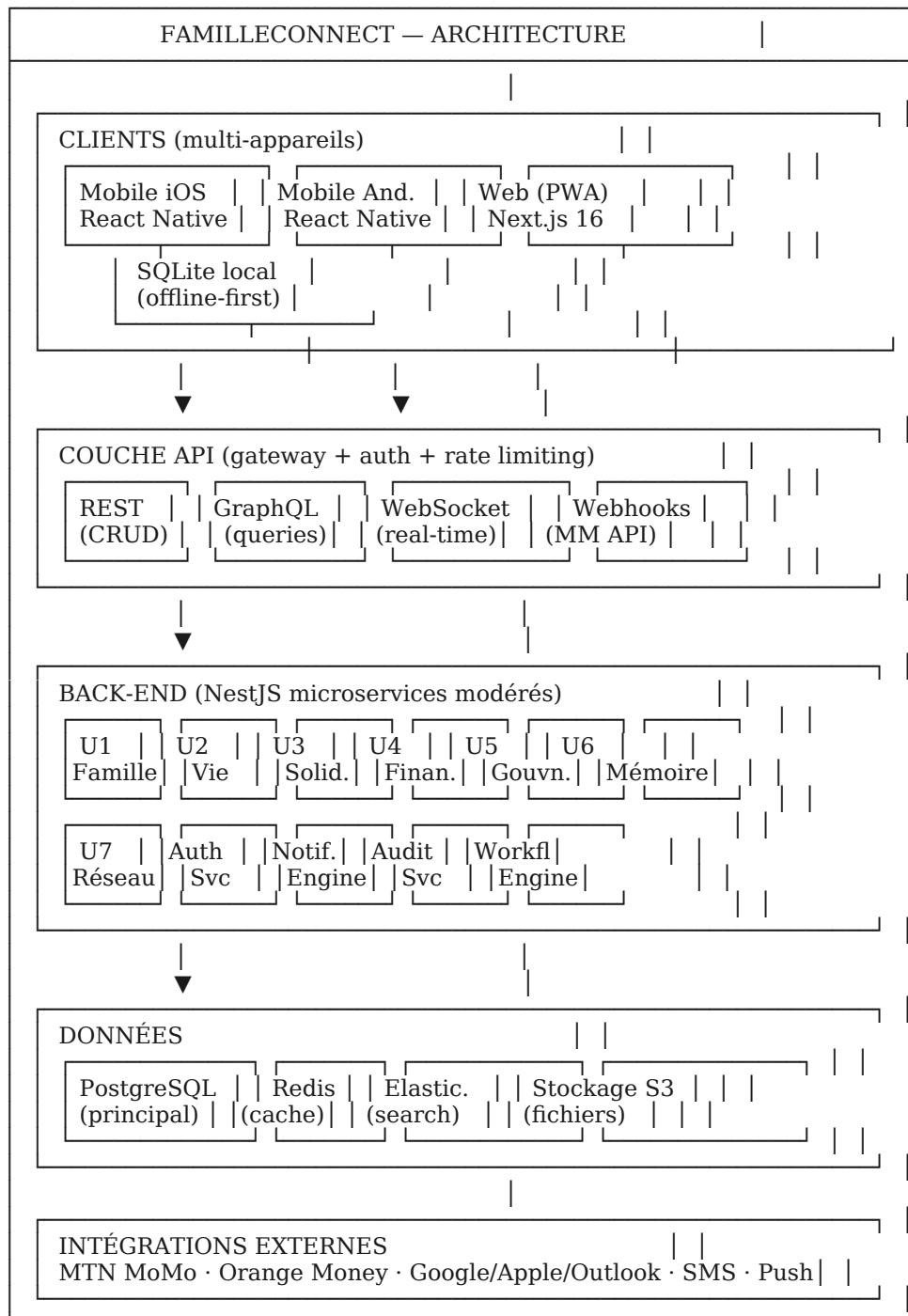


Figure 1 — Architecture globale de la plateforme FamilleConnect

1.3 Flux principaux

Trois flux principaux structurent l'architecture :

- **Flux synchrone (REST/GraphQL)** — pour les opérations CRUD classiques (consultation du registre, mise à jour d'une fiche, soumission d'un vote). Le client appelle l'API, qui interroge le back-end, qui lit ou écrit dans PostgreSQL. Latence cible : < 500 ms pour 95 % des requêtes.
- **Flux temps réel (WebSocket)** — pour les interactions en direct (chat U7, votes en session U5, suivi temps réel d'une collecte U3). Le client ouvre une connexion WebSocket persistante, le serveur pousse les mises à jour. Latence cible : < 2 secondes pour 95 % des messages.
- **Flux asynchrone (queues + webhooks)** — pour les opérations longues ou externes (paiement Mobile Money, envoi de notifications push, génération de rapports, archivage U6). Le client déclenche, le back-end traite en arrière-plan, et notifie le client à la fin. Latence variable selon l'opération.

2. Contraintes techniques du contexte camerounais

L'architecture technique de FamilleConnect est profondément influencée par les réalités camerounaises et africaines. Cinq contraintes majeures structurent les choix techniques.

2.1 Connectivité intermittente

La connectivité Internet au Cameroun est inégale : excellente à Yaoundé et Douala, intermittente en zones rurales, quasi inexistante dans certains villages. Les opérateurs (Orange, MTN, Camtel) offrent une couverture 3G/4G variable, et les coupures ne sont pas rares. Pour FamilleConnect, cela signifie :

- L'application mobile doit fonctionner hors-ligne pour les opérations critiques (consultation du registre, saisie d'un paiement en espèces, consultation des soldes).
- La synchronisation doit être automatique et tolérante aux coupures (reprise là où elle s'est arrêtée).
- Les conflits de synchronisation doivent être gérés (par exemple, deux modifications concurrentes sur la même fiche).
- Les congrès en zone rurale doivent pouvoir utiliser l'application en réseau local, avec synchronisation différée.

2.2 Mobile Money dominant

Au Cameroun, le Mobile Money (MTN MoMo, Orange Money) est le canal de paiement dominant pour les transactions de la vie courante. Les cartes bancaires sont rares, les virements bancaires réservés aux entreprises et aux montants importants. Pour FamilleConnect, cela signifie :

- L'intégration des API MTN MoMo et Orange Money est critique, pas optionnelle.
- L'UX de paiement doit intégrer nativement le parcours Mobile Money (USSD, popup opérateur, confirmation).
- Le rapprochement automatique entre paiements Mobile Money et écritures comptables est essentiel pour la transparence U4.
- Le mode espèces doit être prévu pour les membres sans Mobile Money, avec saisie manuelle par le trésorier.

2.3 Multi-fuseaux diaspora

Les grandes familles camerounaises ont une diaspora répartie sur plusieurs fuseaux horaires : Europe (UTC+1/+2), Amérique du Nord (UTC-5 à -8), Asie (UTC+7 à +9). Pour FamilleConnect, cela signifie :

- Toutes les horaires stockées en UTC, converties en heure locale à l'affichage.
- Le moteur de notifications doit calculer l'heure locale de chaque destinataire à partir de son profil U1.
- Les heures de repos sont personnalisables par membre (par défaut 22h-7h en heure locale).
- Les votes en session (U5) et les congrès hybrides (U2) doivent gérer des participants dans des fuseaux différents, avec une fenêtre de vote synchronisée.

2.4 Coût et pouvoir d'achat

Le modèle économique de FamilleConnect repose sur des abonnements familiaux modiques (10 000 à 25 000 FCFA/an). L'infrastructure ne peut pas représenter un coût élevé par membre. Pour FamilleConnect, cela signifie :

- Privilégier les solutions open-source (PostgreSQL, Redis, NestJS) aux solutions propriétaires payantes.
- Utiliser des services managés économiques (Amazon RDS, DigitalOcean, Hetzner) plutôt que des infrastructures premium.
- Optimiser en continu les coûts de stockage (compression, lifecycle, archive froide).
- Viser un coût d'infrastructure < 500 FCFA par membre et par mois à maturité.

2.5 Données sensibles et réglementation

FamilleConnect gère des données particulièrement sensibles : santé, précarité, conflits familiaux, testaments, finances. La réglementation camerounaise (loi n° 2010/012 sur la cybercriminalité) et les évolutions réglementaires OHADA/CEMAC imposent des obligations. Pour FamilleConnect, cela signifie :

- Chiffrement au repos et en transit pour toutes les données.
- Journalisation immuable des actions sensibles (consultation de dossiers sensibles, validation de dépenses, modifications de rôles).
- Conservation longue durée des archives (U6) avec traçabilité.
- Hébergement envisageable au Cameroun ou en zone CEMAC pour la souveraineté des données, à étudier en Phase 2.
- Conformité aux principes RGPD-like (droit d'accès, rectification, effacement partiel, portabilité).

3. Front-end mobile

3.1 Choix du framework : React Native

Le front-end mobile est développé en React Native, le framework cross-platform de Meta. Ce choix se justifie par plusieurs facteurs :

- **Code partagé iOS/Android** — un seul codebase pour les deux plateformes, ce qui réduit les coûts de développement et de maintenance de moitié par rapport à un développement natif double.
- **Écosystème mature** — React Native est utilisé par Instagram, Facebook, Shopify, Discord, et de nombreuses applications de production. La communauté est vaste, les bibliothèques nombreuses, les retours d'expérience documentés.
- **TypeScript natif** — React Native supporte TypeScript de première classe, ce qui permet un typage strict et une détection des erreurs à la compilation.
- **Hot reload** — le développement est rapide grâce au hot reload, qui affiche les modifications instantanément sans recompilation.
- **Performance suffisante** — pour une application de gestion comme FamilleConnect (pas de jeux, pas de calculs intensifs), la performance de React Native est largement suffisante.
- **Recrutement local** — la communauté React Native est présente au Cameroun, ce qui facilite le recrutement de développeurs.

Les alternatives envisagées (Flutter, native iOS+Android, Ionic) ont été écartées : Flutter pour la taille de l'écosystème encore en croissance, natif pour le coût double, Ionic pour la performance en mode hors-ligne.

3.2 Architecture offline-first

L'architecture offline-first est le pilier du front-end mobile. Elle repose sur quatre composants :

- **Base de données locale SQLite** — chaque appareil dispose d'une base SQLite locale qui réplique un sous-ensemble des données du serveur (registre familial, soldes, prochains événements, conversations en cours). Cette base permet la consultation et la saisie hors-ligne.
- **Moteur de synchronisation** — un moteur dédié gère la synchronisation entre la base locale et le serveur. Il détecte les modifications locales, les envoie au serveur dès que la connexion est disponible, et récupère les modifications distantes. Il gère les conflits (dernière modification gagne, ou résolution manuelle pour les cas complexes).
- **File d'attente des actions** — les actions sensibles (paiement, vote, validation) sont placées dans une file d'attente persistante, et exécutées dès que la connexion est disponible. Le membre reçoit une confirmation visuelle (« En attente de synchronisation ») puis une confirmation définitive (« Synchronisé »).
- **Cache stratégique** — les données fréquemment consultées (sa propre fiche, ses cotisations, ses notifications) sont mises en cache de manière agressive, pour une consultation instantanée même hors-ligne.

3.3 Gestion des notifications push

Les notifications push sont gérées par les services natifs : Firebase Cloud Messaging (FCM) pour Android, Apple Push Notification service (APNs) pour iOS. React Native s'interface avec ces services via des bibliothèques dédiées (react-native-firebase, react-native-push-notification).

Le flux de notification push est le suivant :

- Le back-end déclenche une notification via le moteur de notifications.
- Le moteur appelle FCM/APNs avec le jeton de l'appareil destinataire.
- FCM/APNs délivre la notification à l'appareil.
- Si l'application est en arrière-plan, la notification s'affiche dans le centre de notifications.
- Si l'application est au premier plan, la notification est traitée en interne et affichée dans l'UI.

Les notifications silencieuses (pour réveiller l'application et déclencher une synchronisation) sont également utilisées, avec parcimonie pour préserver la batterie.

3.4 Accessibilité et multi-langue

L'application mobile est conçue pour être accessible à des membres dont la maîtrise du numérique varie, et dont la langue maternelle peut ne pas être le français :

- **Interface en gros caractères** — option pour les membres âgés, avec tailles de police ajustables.
- **Navigation simplifiée** — maximum 3 taps pour atteindre une information clé.
- **Notifications vocales** — pour les patriarches et membres âgés, lecture vocale des annonces critiques via synthèse vocale.
- **Multi-langue** — français (par défaut), anglais, et à terme langues locales (medumba, fe'fe', ghomala', ewondo, duala). L'architecture i18n est en place dès le départ, même si les traductions locales seront ajoutées progressivement.
- **Mode sombre** — pour économiser la batterie et réduire la fatigue oculaire.

3.5 Sécurité mobile

La sécurité mobile repose sur :

- **Authentification biométrique** — Face ID / Touch ID pour le déverrouillage de l'application, en complément du mot de passe.
- **Chiffrement local** — la base SQLite locale est chiffrée (SQLCipher), pour protéger les données en cas de perte ou de vol de l'appareil.
- **Jeton à courte durée** — le jeton d'authentification a une durée de validité courte (1 heure), avec refresh automatique, et est révoqué en cas de déconnexion.
- **Effacement à distance** — en cas de perte de l'appareil, le membre peut révoquer tous ses jetons depuis le web, ce qui empêche l'accès aux données locales (qui restent chiffrées sans la clé).

3.6 Distribution et mises à jour

- **Android** — distribution via Google Play Store et, en complément, via téléchargement direct (APK) pour les appareils sans Google Play (fréquents au Cameroun).
- **iOS** — distribution via Apple App Store, avec revue Apple standard.

- **Mises à jour** — CodePush (Microsoft App Center) pour les mises à jour JavaScript (sans repasser par les stores), mises à jour natives via les stores pour les modifications de code natif.
- **Versioning** — semver, avec politique de support des N-2 versions (les 3 dernières versions mineures sont supportées).

4. Front-end web

4.1 Choix du framework : Next.js 16

Le front-end web est développé en Next.js 16 (App Router), le framework React de Vercel. Ce choix se justifie par :

- **Server-Side Rendering (SSR) et Static Site Generation (SSG)** — pour des performances optimales, notamment pour la diaspora qui peut avoir une connexion faible.
- **App Router** — architecture moderne avec Server Components, Streaming, et Suspense.
- **TypeScript natif** — cohérence avec le mobile React Native et le back-end NestJS.
- **Écosystème** — Vercel pour l'hébergement (optionnel), mais surtout écosystème React complet.
- **SEO** — bien que FamilleConnect ne soit pas une application publique, le SEO peut être utile pour les pages publiques (landing, blog, documentation).
- **PWA** — Next.js supporte nativement les Progressive Web Apps, ce qui permet une expérience hors-ligne légère sur navigateur.

4.2 Architecture web

L'architecture web Next.js suit les bonnes pratiques modernes :

- **App Router** — avec dossiers app/ pour le routage, layouts imbriqués, et Server Components par défaut.
- **Server Components** — pour les pages qui n'ont pas besoin d'interactivité (consultation de registre, lecture d'albums), réduisant le JavaScript envoyé au client.
- **Client Components** — pour les interactions (formulaires, votes, messagerie), avec hydratation sélective.
- **Streaming** — avec Suspense pour les pages longues à charger (albums photo, archives U6), affichant progressivement le contenu.
- **API Routes** — pour les endpoints légers (webhooks Mobile Money, callbacks OAuth), délégués au back-end NestJS pour la logique métier.

4.3 Mode hors-ligne léger (PWA)

Le web ne peut pas offrir un mode hors-ligne aussi riche que le mobile (pas de base SQLite locale), mais il peut offrir un mode hors-ligne léger via PWA :

- **Service Worker** — mise en cache des assets statiques (CSS, JS, images) pour un chargement rapide même hors-ligne.
- **Cache API** — mise en cache des réponses API pour consultation hors-ligne (dernière valeur connue).
- **Background Sync** — file d'attente des actions effectuées hors-ligne, synchronisées au retour de la connexion.
- **Installable** — l'utilisateur peut installer l'application web sur son écran d'accueil, pour une expérience proche du natif.

Ce mode PWA est particulièrement utile pour la diaspora qui préfère le web au mobile, et pour les membres qui utilisent un ordinateur.

4.4 Optimisation diaspora

La diaspora est un public important, avec des contraintes spécifiques :

- **Réseau variable** — la diaspora peut avoir une excellente connexion (Europe) ou une connexion limitée (certains pays). L'application web doit être optimisée pour tous les profils.
- **Latence** — la diaspora est éloignée des serveurs (potentiellement hébergés en Afrique). L'utilisation d'un CDN mondial (Cloudflare, AWS CloudFront) réduit la latence.
- **Multi-langue** — la diaspora peut préférer l'anglais ou la langue du pays d'accueil, en complément du français.
- **Fuseaux horaires** — toutes les heures affichées en heure locale du navigateur, calculée à partir du fuseau du membre.

4.5 Accessibilité web

L'accessibilité web suit les standards WCAG 2.1 AA :

- **Sémantique HTML** — utilisation correcte des balises (header, nav, main, article, aside, footer) pour les lecteurs d'écran.
- **Contrastes** — respect des ratios de contraste pour la lisibilité (texte sur fond, boutons).
- **Navigation au clavier** — toutes les actions atteignables au clavier, avec focus visible.
- **Tailles de police** — tailles relatives (rem, em) pour le zoom utilisateur.

- ARIA — attributs ARIA pour les composants complexes (modales, menus déroulants, onglets).

5. Back-end

5.1 Choix du framework : NestJS

Le back-end est développé en NestJS, le framework Node.js/TypeScript inspiré d'Angular. Ce choix se justifie par :

- **Architecture modulaire** — NestJS encourage une organisation en modules, parfaitement adaptée à la séparation des 7 univers. Chaque univers est un module dédié, avec ses contrôleurs, services, et repositories.
- **TypeScript natif** — typage strict, décorateurs, injection de dépendances, cohérence avec le front-end.
- **Écosystème mature** — NestJS est utilisé par de nombreuses entreprises (Roche, Adidas, Société Générale), avec une documentation riche et une communauté active.
- **Out-of-the-box** — NestJS fournit des solutions pour l'authentification, la validation, la documentation API (Swagger), les tests, le logging, ce qui accélère le développement.
- **Microservices-ready** — NestJS supporte nativement les microservices (TCP, Redis, Kafka, gRPC), ce qui permet une évolution progressive vers une architecture microservices si le monolithe atteint ses limites.

5.2 Architecture en microservices modérés

Plutôt qu'un monolithe strict ou des microservices complets, FamilleConnect adopte une architecture en microservices modérés : un seul déploiement (monolithe), mais une organisation interne en modules fortement découplés, prêts à être extraits en microservices si le besoin s'en fait sentir.

Les modules métiers correspondent aux 7 univers :

- **U1 Famille Module** — gestion du registre, de l'arbre, des branches, des rôles.
- **U2 Vie Familiale Module** — gestion des événements, du calendrier, du moteur workflow.
- **U3 Solidarité Module** — gestion des dossiers de solidarité, des collectes, du suivi.
- **U4 Finances Module** — gestion de la caisse, des cotisations, de la comptabilité, des paiements Mobile Money.
- **U5 Gouvernance Module** — gestion des votes, des comités, des élections, des PV.
- **U6 Mémoire Module** — gestion des archives, des albums, des interviews, des traditions.

- **U7 Réseau Module** — gestion des compétences, de l'entraide, du mentorat, de la diaspora.

Les modules transverses (qui ne sont pas rattachés à un univers mais servent tous) :

- **Auth Service** — authentification, autorisation, gestion des jetons, 2FA.
- **Notification Engine** — moteur de notifications multi-canal, anti-saturation, préférences.
- **Workflow Engine** — moteur de workflows configurable, utilisé principalement par U2.
- **Audit Service** — journalisation immuable des actions sensibles.
- **File Storage Service** — gestion des fichiers (photos, documents, vidéos), délégation au stockage S3.
- **Search Service** — interface avec Elasticsearch pour la recherche full-text.

5.3 Workflow Engine

Le Workflow Engine est un composant critique, utilisé principalement par l'Univers 2 (moteur événementiel automatisé). Il est responsable de l'exécution des workflows configurables (décès, mariage, congrès, etc.), avec leurs étapes, leurs déclencheurs, et leurs critères de complétion.

Le moteur implémente :

- **Définition des workflows** — en configuration (YAML ou JSON), avec étapes, déclencheurs, responsables, critères de complétion.
- **Instances de workflows** — chaque déclenchement crée une instance, suivie en base de données.
- **Exécution des étapes** — le moteur exécute les étapes selon les déclencheurs, en notifiant les responsables, en surveillant les critères, et en passant à l'étape suivante.
- **Gestion des retards** — le moteur détecte les retards (J+N sans action) et déclenche les relances.
- **Audit** — toutes les actions du moteur sont journalisées, pour audit et optimisation.

Le Workflow Engine est conçu pour être configurable sans modification de code : un administrateur (patriarche, conseil) peut modifier un workflow via une interface dédiée, et le moteur exécute la nouvelle version.

5.4 Notification Engine

Le Notification Engine est le composant qui implémente le catalogue des 121 notifications. Il reçoit les événements déclencheurs depuis les modules métiers, évalue la priorité, le canal, le ciblage, et l'anti-saturation, puis envoie la notification via le canal approprié.

Le moteur implémente :

- Catalogue des notifications — défini en configuration, avec déclencheur, canal, priorité, destinataires, texte type.
- Évaluation de la priorité — chaque notification a un niveau (Critique, Urgent, Standard, Information, Silencieux) qui détermine son comportement.
- Ciblage — calcul des destinataires à partir du périmètre (branche, génération, diaspora, rôle).
- Respect des fuseaux horaires — calcul de l'heure locale de chaque destinataire, diffraction si hors heures de repos.
- Anti-saturation — plafonds quotidiens et hebdomadaires, digests, regroupement.
- Multi-canal — push (FCM/APNs), SMS (passerelle), vocal (service de synthèse vocale), e-mail (SMTP).
- Journalisation — toutes les notifications envoyées sont journalisées pour audit et optimisation.

5.5 Auth Service

L'Auth Service gère l'authentification et l'autorisation :

- **Authentification** — par mot de passe + 2FA (SMS OTP) pour les opérations sensibles. Possibilité d'authentification biométrique côté client.
- **Jetons JWT** — jetons d'accès à courte durée (1 heure), jetons de refresh à durée moyenne (30 jours), révocables.
- **RBAC** — Role-Based Access Control, avec 5 rôles principaux (patriarche, conseil, chef de branche, membre adulte, membre junior) et permissions par univers.
- **Multi-appareils** — un membre peut avoir plusieurs appareils connectés, avec gestion centralisée et révocation.
- **OAuth** — possibilité de se connecter avec Google ou Apple (pour la diaspora), en complément du compte FamilleConnect.

6. API et interfaces

6.1 Trois types d'API

FamilleConnect expose trois types d'API, adaptés à différents besoins :

- **REST API** — pour les opérations CRUD classiques (consultation, création, modification, suppression). Format JSON, norme OpenAPI 3.0 pour la documentation.

- **GraphQL API** — pour les requêtes complexes (arbre généalogique, recherche multi-critères, agrégations). Permet au client de demander exactement les champs dont il a besoin, évitant le sur-fetching et le sous-fetching.
- **WebSocket API** — pour le temps réel (messaging U7, votes en session U5, suivi temps réel d'une collecte U3, notifications push en direct).

6.2 REST API

La REST API est l'interface principale pour les opérations CRUD. Elle suit les conventions REST standard :

- **Verbes HTTP** — GET (lecture), POST (création), PUT/PATCH (modification), DELETE (suppression logique).
- **Ressources** — organisées par univers (/api/v1/famille/membres, /api/v1/finances/cotisations, etc.).
- **Versioning** — préfixe /api/v1/, avec stratégie de dépréciation pour les évolutions majeures.
- **Pagination** — pagination par curseur pour les grandes listes, avec métadonnées (total, hasNext, cursor).
- **Filtrage et tri** — paramètres de query string (?branche=nord&statut=actif&sort=-dateCreation).
- **Erreurs** — format standardisé avec code, message, et détails.
- **Documentation** — Swagger/OpenAPI 3.0 généré automatiquement à partir des décorateurs NestJS, accessible sur /api/docs.

6.3 GraphQL API

La GraphQL API complète la REST API pour les cas nécessitant des requêtes complexes. Elle est particulièrement adaptée à :

- **Arbre généalogique** — récupération d'un membre avec ses parents, enfants, conjoints, sur plusieurs générations, en une seule requête.
- **Tableau de bord** — agrégation d'indicateurs provenant de plusieurs univers (prochains événements U2, cotisations U4, votes en cours U5).
- **Recherche multi-critères** — recherche combinée sur membres, événements, archives, avec filtrage et tri.

La GraphQL API utilise Apollo Server intégré à NestJS, avec :

- **Schema-first** — schéma défini en SDL (Schema Definition Language), résolveurs en TypeScript.

- **DataLoader** — pour éviter le problème N+1 (batching des requêtes base de données).
- **Persisted queries** — pour réduire la taille des requêtes et améliorer la sécurité.
- **Rate limiting** — limitation du nombre de requêtes par minute et par client.

6.4 WebSocket API

La WebSocket API gère le temps réel. Elle utilise Socket.io (côté serveur) et les bibliothèques client dédiées (côté mobile et web). Les cas d'usage sont :

- **Messagerie U7** — envoi et réception de messages en temps réel, avec accusés de lecture.
- **Votes en session U5** — diffusion en direct des résultats pendant un vote en assemblée.
- **Suivi temps réel U3** — mise à jour en direct du montant collecté pendant une collecte de solidarité.
- **Notifications push en direct** — si l'application est ouverte, les notifications sont affichées en direct via WebSocket, sans passer par FCM/APNs.
- **Présence** — indicateur de membres en ligne (pour la messagerie).

La WebSocket API gère la reconnexion automatique en cas de coupure, avec reprise des messages manqués (buffer de 5 minutes).

6.5 Webhooks

Les webhooks sont utilisés pour les intégrations externes qui nécessitent une notification asynchrone :

- **MTN MoMo** — notification de paiement reçu, notification de paiement échoué.
- **Orange Money** — notification de paiement reçu, notification de paiement échoué.
- **Calendriers externes** — synchronisation bidirectionnelle avec Google Calendar, Apple Calendar, Outlook.
- **Services tiers** — intégration avec des services externes (visioconférence, génération de PDF, IA).

Les webhooks sont signés (HMAC SHA-256) pour garantir l'authenticité, et idempotents (traitage unique même en cas de retransmission).

6.6 Rate limiting et sécurité API

- **Rate limiting** — limitation par IP et par jeton (par défaut 100 requêtes par minute pour un utilisateur authentifié, 10 pour un anonyme).
- **CORS** — politique stricte, avec liste blanche des domaines autorisés.
- **Helmet** — middleware de sécurisation des en-têtes HTTP.

- **Validation** — validation systématique des entrées (class-validator en NestJS), avec rejet des payloads invalides.
- **Sanitization** — nettoyage des entrées pour prévenir les injections (SQL, NoSQL, XSS).

7. Base de données

7.1 Vue d'ensemble

La couche de données de FamilleConnect combine plusieurs technologies complémentaires, chacune adaptée à un besoin spécifique :

- **PostgreSQL** — base de données principale (relationnelle, ACID, immutabilité).
- **Redis** — cache distribué et file de messages.
- **Elasticsearch** — moteur de recherche full-text (pour U6).
- **Stockage objet S3-compatible** — fichiers (photos, documents, vidéos).
- **Time-series (optionnel)** — pour les métriques et logs détaillés.

7.2 PostgreSQL — base principale

PostgreSQL est la base de données principale, choisie pour :

- **ACID** — transactions atomiques, cohérentes, isolées, durables, essentielles pour les opérations financières U4.
- **Immutabilité** — capacité à implémenter un journal immuable (U3, U4, U5) via tables d'audit et triggers.
- **Maturité** — PostgreSQL est une base mature, stable, avec une communauté active et des fonctionnalités avancées (JSONB, full-text search, partitions).
- **Open-source** — pas de coût de licence, ce qui correspond au modèle économique.
- **Scalabilité** — réplication en lecture, partitionnement, sharding (à maturité).

7.2.1 Schéma de données

Le schéma PostgreSQL est organisé par univers (schemas PostgreSQL dédiés), avec des tables pour chaque entité principale décrite dans les spécifications :

- **famille schema** — membres, branches, liens, roles, adresses, audit_logs
- **vie_familiale schema** — evenements, etapes_workflow, participants, contributions, documents, messages

- **solidarite schema** — dossiers_solidarite, collectes, contributions_solidarite, situations_suivies, aides_sociales, justificatifs
- **finances schema** — caisses, cotisations, paiements, ecritures_comptables, justificatifs_financiers, budgets
- **gouvernance schema** — votes, comites, elections, mandats, reglement_interieur, proces_verbaux, recours
- **memoire schema** — documents_archives, albums, temoignages, evenements_historiques, archives, traditions, index_recherche
- **reseau schema** — competences, services, mentorats, opportunités, relais_diaspora, recommandations, mises_en_relation

7.2.2 Immutabilité

L'immutabilité est implémentée via :

- **Tables d'audit** — chaque table principale a une table d'audit associée, qui enregistre toutes les modifications (INSERT, UPDATE, DELETE) avec auteur, date, avant, après.
- **Triggers** — des triggers PostgreSQL capturent automatiquement les modifications et les écrivent dans les tables d'audit.
- **Soft delete** — les suppressions sont logiques (colonne deleted_at), jamais physiques, sauf exception validée par le conseil.
- **Journal compensatoire** — pour les corrections d'écritures comptables (U4), une écriture compensatoire est créée, l'originale n'est jamais modifiée.

7.2.3 Réplication

- **Réplication en lecture** — un réplica en lecture pour les requêtes lourdes (rapports, agrégations, recherche), le primaire pour les écritures.
- **Sauvegarde continue** — WAL (Write-Ahead Log) archivé pour restauration point-in-time.
- **Réplication multi-zones** — en Phase 3, réplication vers une zone secondaire pour la continuité d'activité.

7.3 Redis — cache et messages

Redis est utilisé pour plusieurs cas :

- **Cache** — cache des résultats de requêtes fréquents (registre familial, soldes, prochains événements), avec TTL configurable.
- **Sessions** — stockage des sessions JWT (révocables individuellement).
- **Rate limiting** — compteur de requêtes par IP et par jeton.

- **File de messages** — pour les tâches asynchrones (envoi de notifications, génération de rapports, synchronisation Mobile Money).
- **Locks distribués** — pour les opérations sensibles (double validation, transition de rôles).
- **WebSocket adapter** — pour la synchronisation entre instances Socket.io.

7.4 Elasticsearch — recherche full-text

Elasticsearch est utilisé pour la recherche full-text, principalement par l'Univers 6 (Mémoire) et l'Univers 7 (Réseau) :

- Recherche dans les archives U6 — documents, témoignages, traditions, albums.
- Recherche de compétences U7 — recherche par mot-clé, catégorie, localisation.
- Recherche de membres U1 — par nom, branche, profession, lieu.
- Suggestions — autocomplétion, recherches populaires.

Les données sont indexées en temps réel depuis PostgreSQL (via un connecteur de changement de données - CDC), ce qui garantit la cohérence entre la base principale et le moteur de recherche.

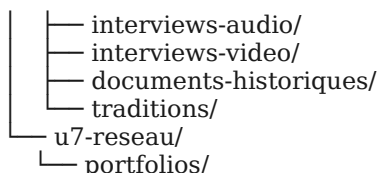
7.5 Stockage objet S3-compatible

Le stockage des fichiers (photos, documents, vidéos) utilise un stockage objet S3-compatible :

- **AWS S3** — en Phase 1, pour la simplicité et la maturité.
- **MinIO (auto-hébergé)** — en Phase 2 ou 3, pour réduire les coûts et gagner en souveraineté.
- **Triple stockage** — pour les archives U6 (voir spécifications U6), chaque fichier est stocké sur trois supports indépendants : S3 principal, sauvegarde externalisée (autre région ou autre fournisseur), archive froide (glacier ou équivalent).

Les fichiers sont organisés par univers et par type :

```
s3://familleconnect/
├── u1-membres/
│   ├── photos-profil/
│   └── documents-identite/
├── u2-evenements/
│   ├── albums/
│   └── programmes/
├── u3-solidarite/
│   └── justificatifs/
├── u4-finances/
│   ├── justificatifs-depenses/
│   └── releves/
├── u5-gouvernance/
│   ├── pv/
│   └── reglement/
└── u6-memoire/
```



Chaque fichier a une URL signée (à durée de validité limitée) pour l'accès, ce qui garantit que seuls les membres autorisés peuvent le consulter.

7.6 Stratégie de sauvegarde

La stratégie de sauvegarde est critique, notamment pour U6 (Mémoire) :

- **Sauvegarde PostgreSQL** — quotidienne (dump complet), continue (WAL), rétention 30 jours en ligne, 1 an en archive froide.
- **Sauvegarde Redis** — snapshot toutes les 6 heures, rétention 7 jours.
- **Sauvegarde Elasticsearch** — snapshot quotidien, rétention 30 jours.
- **Sauvegarde stockage S3** — réplication cross-region, versioning activé, lifecycle vers archive froide après 90 jours.
- **Sauvegarde externe trimestrielle** — export complet chiffré vers un lieu géographiquement distant, pour la continuité d'activité.
- **Tests de restauration** — au moins une fois par an, test complet de restauration sur environnement de test.

8. Hébergement et infrastructure

8.1 Choix du fournisseur cloud

Le choix du fournisseur cloud est un compromis entre coût, maturité, disponibilité régionale, et facilité d'utilisation. Trois options sont envisagées :

- **AWS (Amazon Web Services)** — maturité, services managés, présence en Afrique du Sud (Cape Town). Coût plus élevé.
- **Azure (Microsoft)** — présence en Afrique du Sud, services managés, intégration Microsoft. Coût similaire à AWS.
- **Hetzner Cloud** — coût très compétitif, datacenters en Europe (Allemagne, Finlande), mais pas en Afrique. Idéal pour le coût, moins pour la latence.

Recommandation Phase 1 : AWS pour la maturité et la présence africaine, avec une optimisation des coûts via instances réservées et spots. Migration vers Hetzner en Phase 3 si les coûts deviennent un enjeu, avec un CDN pour réduire la latence.

8.2 Architecture d'hébergement

L'architecture d'hébergement est multi-zones pour la haute disponibilité :

- **Zone principale** — Afrique du Sud (AWS af-south-1), pour la proximité avec le Cameroun et la latence réduite.
- **Zone secondaire** — Europe (AWS eu-west-3 pour Paris, eu-central-1 pour Francfort), pour la diaspora et la redondance.
- **CDN mondial** — Cloudflare ou AWS CloudFront, pour la diffusion des assets statiques et la réduction de latence pour la diaspora.

8.3 Composants d'infrastructure

- **Load balancer** — AWS Application Load Balancer, avec SSL termination et routing par chemin.
- **Compute** — AWS ECS (Elastic Container Service) avec Fargate, pour l'exécution des conteneurs sans gestion de serveurs.
- **Base de données** — Amazon RDS for PostgreSQL, avec réplication multi-AZ.
- **Cache** — Amazon ElastiCache for Redis.
- **Recherche** — Amazon Elasticsearch Service (ou OpenSearch).
- **Stockage objet** — Amazon S3, avec lifecycle vers Glacier pour l'archive froide.
- **CDN** — Amazon CloudFront (ou Cloudflare).
- **DNS** — Amazon Route 53.
- **Monitoring** — Amazon CloudWatch + Datadog (pour la richesse des dashboards).

8.4 Conteneurisation et orchestration

- **Docker** — chaque composant (back-end, front-end web, workers) est conteneurisé.
- **AWS ECS avec Fargate** — orchestration sans gestion de serveurs, idéale pour la Phase 1 et 2.
- **Kubernetes (optionnel Phase 3)** — si la scalabilité le justifie, migration vers EKS (Amazon Elastic Kubernetes Service).

8.5 Plan de continuité d'activité

- **RTO (Recovery Time Objective)** — 4 heures (temps maximal pour restaurer le service après un incident majeur).
- **RPO (Recovery Point Objective)** — 1 heure (perte maximale de données acceptable).

- **Multi-AZ** — déploiement sur plusieurs zones de disponibilité pour tolérer la perte d'une zone.
- **Multi-région (Phase 3)** — déploiement sur deux régions pour tolérer la perte d'une région.
- **Sauvegarde externe** — export chiffré vers un fournisseur tiers, pour tolérer la perte du fournisseur principal.
- **Plan de communication** — en cas d'incident, communication transparente aux familles via un canal de secours (e-mail, page de statut).

8.6 Hébergement local envisageable

En Phase 3, un hébergement local au Cameroun (ou en zone CEMAC) peut être envisagé pour :

- **Souveraineté des données** — respect de la réglementation locale et de la préférence nationale.
- **Latence réduite** — pour les membres au Cameroun, latence plus faible.
- **Coût** — potentiellement plus bas à terme, avec un fournisseur local (Camtel, MTN Business).

Les défis : maturité des datacenters locaux, fiabilité électrique, bande passante internationale. À évaluer en Phase 2 avec un proof of concept.

9. Sécurité technique transverse

9.1 Authentification

- **Mot de passe** — politique de complexité (8 caractères minimum, majuscule, minuscule, chiffre), hashage bcrypt (coût 12).
- **2FA (Two-Factor Authentication)** — obligatoire pour les opérations sensibles (paiement > 50 000 FCFA, vote, modification de rôles). Code OTP par SMS, validité 5 minutes.
- **Biométrie** — Face ID / Touch ID côté mobile, en complément du mot de passe.
- **OAuth** — connexion avec Google ou Apple (pour la diaspora), en complément du compte FamilleConnect.
- **Récupération de compte** — procédure sécurisée avec vérification d'identité (SMS + e-mail + validation par un chef de branche).

9.2 Autorisation

- **RBAC (Role-Based Access Control)** — 5 rôles principaux (patriarche, conseil, chef de branche, membre adulte, membre junior), avec permissions par univers et par action.
- **ABAC (Attribute-Based Access Control)** — pour les cas avancés (par exemple, accès aux dossiers de sa branche uniquement), basé sur les attributs du membre (branche, génération, localisation).
- **Principe du moindre privilège** — chaque rôle a les permissions strictement nécessaires, pas plus.
- **Audit des accès** — toutes les accès à des données sensibles (dossiers de santé, testaments, dossiers de médiation) sont journalisés.

9.3 Chiffrement

- **En transit** — TLS 1.3 pour toutes les communications (API, WebSocket, webhooks).
- **Au repos** — chiffrement PostgreSQL (AES-256), chiffrement S3 (AES-256), chiffrement Redis.
- **Mobile** — base SQLite locale chiffrée (SQLCipher, AES-256).
- **Sauvegardes** — chiffrement des exports et des archives froides.
- **Gestion des clés** — AWS KMS (Key Management Service) pour la gestion centralisée, rotation automatique.

9.4 Conformité RGPD-like

- **Droit d'accès** — un membre peut consulter toutes les données le concernant.
- **Droit de rectification** — un membre peut rectifier ses données personnelles.
- **Droit à l'effacement partiel** — un membre peut demander l'effacement de certaines données (sous réserve des obligations de mémoire U6).
- **Portabilité** — un membre peut exporter ses données dans un format standard (JSON).
- **Consentement** — consentement explicite pour la collecte et le traitement des données sensibles (santé, précarité).
- **Journal des consultations** — consultation par le membre des accès à ses données sensibles.

9.5 Gestion des secrets

- **AWS Secrets Manager** — stockage centralisé des secrets (mots de passe base de données, clés API, certificats).
- **Rotation automatique** — rotation des mots de passe base de données tous les 90 jours.

- **Pas de secrets dans le code** — tous les secrets sont en Secrets Manager, jamais dans le code source ou les variables d'environnement.
- **Accès restreint** — seuls les composants qui en ont besoin accèdent aux secrets, avec audit.

10. Performance et scalabilité

10.1 Stratégie de cache

- **Cache Redis** — pour les données fréquemment consultées (registre familial, soldes, prochains événements), avec TTL configurable.
- **Cache HTTP** — en-têtes Cache-Control pour les assets statiques, ETag pour les réponses API.
- **Cache CDN** — mise en cache des assets statiques et des pages web sur le CDN mondial.
- **Cache mobile** — base SQLite locale pour la consultation hors-ligne.
- **Invalidation** — invalidation automatique du cache lors des modifications, avec stratégie par entité.

10.2 Optimisation des requêtes

- **Index PostgreSQL** — index sur les colonnes fréquemment filtrées (branche, statut, date).
- **Partitionnement** — partitionnement des tables volumineuses (écritures comptables, notifications) par mois ou par année.
- **Requêtes optimisées** — utilisation de EXPLAIN ANALYZE pour identifier les requêtes lentes, optimisation continue.
- **N+1** — utilisation de DataLoader (GraphQL) et de relations eagerly loaded (REST) pour éviter le problème N+1.
- **Pagination** — pagination par curseur pour les grandes listes, jamais de OFFSET.

10.3 Scalabilité horizontale

- **Stateless** — le back-end est stateless (pas de session en mémoire), ce qui permet la scalabilité horizontale.
- **Auto-scaling** — AWS ECS avec auto-scaling basé sur la CPU et la mémoire.
- **Réplication en lecture** — PostgreSQL avec réplica en lecture pour les requêtes lourdes.
- **CDN** — téléchargement des assets statiques vers le CDN.

- **Workers asynchrones** — tâches lourdes (génération de rapports, envoi massif de notifications) déléguées à des workers dédiés.

10.4 Gestion des pics

- **Décès simultanés** — en cas de décès dans plusieurs familles, pic de notifications. Le moteur de notifications est conçu pour absorber 10x le trafic normal.
- **Congrès** — pic de connexion lors d'un congrès (plusieurs centaines de membres connectés simultanément). Auto-scaling réactif.
- **Votes** — pic de requêtes lors de la clôture d'un vote. Mise en cache des résultats partiels.
- **Black Friday Mobile Money** — pic de paiements en fin de mois (jours de paie). File d'attente pour absorber.

10.5 Monitoring

- **Métriques** — Datadog pour les métriques applicatives (latence, débit, erreurs), CloudWatch pour les métriques infrastructure.
- **Logs** — logs centralisés (ELK ou Datadog Logs), avec rétention 90 jours en ligne, 1 an en archive.
- **Tracing** — distributed tracing (Datadog APM) pour identifier les goulots d'étranglement.
- **Alerting** — alertes en cas d'anomalie (latence > seuil, taux d'erreur > seuil, indisponibilité).
- **Tableau de bord** — tableau de bord temps réel pour l'équipe technique, avec SLA et SLO.

11. Intégrations externes

11.1 MTN MoMo API

- **API** — MTN MoMo Open API (REST), avec authentification OAuth 2.0.
- **Endpoints** — création de paiement (collection), vérification de statut, notification de paiement (webhook).
- **Webhooks** — réception des notifications de paiement reçu, paiement échoué, en temps réel.
- **Rapprochement** — rapprochement automatique entre les webhooks et les écritures comptables U4.
- **Mode sandbox** — pour les tests en environnement de développement.

11.2 Orange Money API

- **API** — Orange Money Web Payment (REST), avec authentification par clé API.
- **Endpoints** — création de paiement, vérification de statut, notification de paiement (webhook).
- **Webhooks** — réception des notifications de paiement, similaire à MTN MoMo.
- **Rapprochement** — rapprochement automatique, similaire à MTN MoMo.

11.3 Calendriers externes

- **Google Calendar** — synchronisation bidirectionnelle via Google Calendar API (OAuth 2.0).
- **Apple Calendar** — export iCal (read-only), pas de synchronisation bidirectionnelle (limitation d'Apple).
- **Outlook** — synchronisation bidirectionnelle via Microsoft Graph API (OAuth 2.0).
- **iCal générique** — export iCal public (read-only) pour tout calendrier compatible.

11.4 SMS gateway

- **Fournisseur** — Twilio (international), ou passerelle locale (Africa's Talking, Cammob) pour le Cameroun.
- **API** — REST, avec authentification par clé API.
- **Coût** — 5 à 15 FCFA par SMS au Cameroun, variable selon le fournisseur.
- **Usage** — notifications critiques (décès), 2FA, relances de cotisation.

11.5 E-mail

- **Fournisseur** — Amazon SES (Simple Email Service), ou SendGrid pour la délivrabilité.
- **API** — REST, avec authentification par clé API.
- **Coût** — quasi nul (0,10 USD pour 1000 e-mails).
- **Usage** — rapports mensuels, bilans annuels, notifications diaspora.

11.6 Push notifications

- **FCM (Firebase Cloud Messaging)** — pour Android, gratuit.
- **APNs (Apple Push Notification service)** — pour iOS, gratuit.
- **Web Push** — pour les navigateurs supportant les Web Push Notifications.

11.7 Visioconférence (Phase 2)

- **Intégration** — Zoom API, Google Meet API, ou Jitsi Meet (open-source, auto-hébergeable).
- **Usage** — congrès hybrides (U2), assemblées avec diaspora, mentorat à distance (U7).

12. Stack technique récapitulative

Le tableau ci-dessous synthétise l'ensemble des choix technologiques par couche, avec justification et alternative envisagée.

Tableau 1 — Stack technique récapitulative

Couche	Technologie	Justification	Alternative
Mobile	React Native + TypeScript	Code partagé iOS/Android, écosystème mature	Flutter
Web	Next.js 16 + TypeScript + Tailwind	SSR/SSG, performance, PWA	Remix, Nuxt
Back-end	NestJS + TypeScript	Architecture modulaire, écosystème	Express, Fastify
API REST	NestJS + Swagger	Natif NestJS, OpenAPI 3.0	Express + Swagger
API GraphQL	Apollo Server + NestJS	Schema-first, DataLoader	yoga, mercurius
WebSocket	Socket.io	Reconnexion auto, rooms	native WS, Pusher
Base principale	PostgreSQL 15+	ACID, immutabilité, maturité	MySQL, MariaDB
Cache	Redis 7+	Performance, polyvalence	Memcached
Recherche	Elasticsearch 8+ / OpenSearch	Full-text, agrégations	Meilisearch, Typesense
Stockage objet	Amazon S3 / MinIO	Maturité, coût, API standard	Google Cloud Storage
ORM	Prisma	Type-safe, migrations	TypeORM, Sequelize
File de messages	Redis (BullMQ)	Simplicité, intégration Redis	RabbitMQ, Kafka
Conteneurs	Docker	Standard de fait	Podman
Orchestration	AWS ECS + Fargate	Simplicité, serverless	Kubernetes (EKS)
Cloud	AWS	Maturité, présence Afrique	Azure, Hetzner
CDN	Cloudflare ou CloudFront	Performance diaspora	Akamai
DNS	Route 53	Intégration AWS	Cloudflare DNS
Monitoring	Datadog + CloudWatch	Richesse dashboards	Grafana + Prometheus
Logs	Datadog Logs	Centralisation	ELK
CI/CD	GitHub Actions	Intégration GitHub	GitLab CI, CircleCI
Secrets	AWS Secrets Manager	Intégration AWS	HashiCorp Vault

Couche	Technologie	Justification	Alternative
SMS	Twilio / Africa's Talking	Couverture Cameroun	Cammob
E-mail	Amazon SES	Coût, intégration AWS	SendGrid, Postmark
Push mobile	FCM + APNs	Standard	—
Tests	Jest + Playwright	Couverture unit + e2e	Vitest, Cypress

13. Coûts d'infrastructure indicatifs

13.1 Phase 1 — MVP (5-10 familles, 1 500-3 000 membres)

Tableau 2 — Coûts Phase 1 (MVP)

Poste	Service	Coût mensuel (FCFA)
Compute (ECS Fargate)	2-3 conteneurs back-end + 1 web	200 000 – 350 000
Base de données (RDS)	PostgreSQL db.t3.medium + réplique	250 000 – 350 000
Cache (ElastiCache)	Redis cache.t3.small	80 000 – 120 000
Stockage (S3)	50-100 Go	20 000 – 40 000
CDN (CloudFront)	50-100 Go de trafic	50 000 – 100 000
SMS	1 000-3 000 SMS	50 000 – 150 000
E-mail (SES)	10 000-30 000 e-mails	5 000 – 15 000
Monitoring (Datadog)	Plan Pro, 5-10 hosts	150 000 – 250 000
Divers	DNS, secrets, logs	50 000 – 100 000
Total Phase 1		855 000 – 1 475 000

13.2 Phase 2 — Enrichissement (30-100 familles, 9 000-30 000 membres)

Tableau 3 — Coûts Phase 2

Poste	Coût mensuel (FCFA)
Compute (ECS auto-scaling)	500 000 – 900 000
Base de données (RDS plus puissante)	500 000 – 800 000
Cache (Redis plus puissant)	150 000 – 250 000
Elasticsearch (OpenSearch)	300 000 – 500 000
Stockage (S3, 500 Go – 2 To)	150 000 – 500 000
CDN (200-500 Go de trafic)	200 000 – 400 000
SMS (10 000-30 000 SMS)	500 000 – 1 500 000

Poste	Coût mensuel (FCFA)
E-mail	20 000 – 50 000
Monitoring	250 000 – 400 000
Divers	100 000 – 200 000
Total Phase 2	2,5 – 5,5 millions

13.3 Phase 3 — Maturité (500-2 000 familles, 150 000-600 000 membres)

Tableau 4 — Coûts Phase 3

Poste	Coût mensuel (FCFA)
Compute (ECS auto-scaling, 10-20 conteneurs)	2 – 4 millions
Base de données (RDS + read replicas)	2 – 4 millions
Cache (Redis cluster)	500 000 – 1 million
Elasticsearch (cluster)	1 – 2 millions
Stockage (S3 + Glacier, 10-50 To)	500 000 – 2 millions
CDN (1-5 To de trafic)	500 000 – 2 millions
SMS (100 000-500 000 SMS)	5 – 25 millions
E-mail	100 000 – 300 000
Monitoring	500 000 – 1 million
Divers	300 000 – 700 000
Total Phase 3	12 – 42 millions

13.4 Coût par membre

Tableau 5 — Coût par membre et par mois

Phase	Membres	Coût mensuel total	Coût par membre/mois
Phase 1	1 500-3 000	1 – 1,5 M FCFA	350 – 1 000 FCFA
Phase 2	9 000-30 000	2,5 – 5,5 M FCFA	180 – 600 FCFA
Phase 3	150 000-600 000	12 – 42 M FCFA	70 – 280 FCFA

L'objectif de coût < 500 FCFA par membre et par mois est atteint dès la Phase 2, et optimisé en Phase 3.

14. Risques techniques et mitigations

Le tableau ci-dessous présente les 10 principaux risques techniques et leurs stratégies de mitigation.

Tableau 6 — Top 10 des risques techniques

#	Risque	Probabilité	Impact	Mitigation
1	Indisponibilité MTN MoMo / Orange Money	Moyenne	Élevé	Mode espèces de secours, file d'attente, communication transparente
2	Perte de données (PostgreSQL)	Faible	Critique	Sauvegardes quotidiennes + WAL, réplication multi-AZ, tests de restauration annuels
3	Latence excessive pour la diaspora	Moyenne	Moyen	CDN mondial, serveurs multi-régions, optimisation des payloads
4	Faible de sécurité (injection, XSS)	Moyenne	Critique	Validation systématique, sanitization, audits réguliers, bug bounty
5	Saturation des notifications (spam)	Moyenne	Élevé	Anti-saturation, plafonds, digests, préférences membre
6	Dépendance AWS (vendor lock-in)	Faible	Moyen	Architecture portable (Docker, PostgreSQL), sauvegardes externes
7	Coût d'infrastructure sous-estimé	Moyenne	Élevé	Monitoring des coûts, optimisation continue, alertes budget
8	Recrutement technique au Cameroun	Moyenne	Élevé	Formation interne, partenariats écoles, télétravail diaspora
9	Évolution réglementaire (RGPD, OHADA)	Faible	Moyen	Veille juridique, conformité by design, hébergement local envisageable
10	Obsolescence des formats (U6)	Faible	Élevé	Formats durables (PDF/A, JPEG, MP3, MP4), migration périodique

15. Plan de déploiement et CI/CD

15.1 Environnements

- **Développement** — environnement local des développeurs, avec Docker Compose pour les dépendances (PostgreSQL, Redis, etc.).
- **Staging** — environnement de pré-production, miroir de la production, pour les tests d'intégration et les validations.
- **Production** — environnement live, accessible aux familles.

15.2 CI/CD

- **GitHub Actions** — pipelines CI/CD intégrés à GitHub.

- **Pipeline** — lint → tests unitaires → build → tests d'intégration → déploiement staging → tests e2e → déploiement production (manuel).
- **Tests automatisés** — Jest pour les tests unitaires (couverture > 80 %), Playwright pour les tests e2e, k6 pour les tests de charge.
- **Blue-green deploys** — déploiement sans interruption, avec bascule instantanée et rollback en cas de problème.
- **Feature flags** — activation/désactivation de fonctionnalités par environnement ou par famille, via LaunchDarkly ou solution interne.

15.3 Monitoring post-déploiement

- **Surveillance** — monitoring renforcé pendant les 24 heures suivant un déploiement.
- **Alertes** — alertes en cas d'augmentation du taux d'erreur ou de latence.
- **Rollback** — procédure de rollback automatisée, déclenchable en moins de 5 minutes.

16. Conclusion

L'architecture technique proposée pour FamilleConnect repose sur des choix pragmatiques, adaptés aux contraintes du contexte camerounais et aux exigences des 7 univers. Elle privilégie la maturité (React Native, Next.js, NestJS, PostgreSQL), l'open-source (coût maîtrisé), et l'offline-first (connectivité intermittente), tout en garantissant la sécurité (chiffrement, 2FA, immutabilité) et la scalabilité (multi-zones, auto-scaling).

Les cinq piliers — front-end mobile, front-end web, back-end, API, base de données, hébergement — forment un ensemble cohérent, où chaque choix se justifie par les contraintes du projet et les meilleures pratiques de l'industrie. La stack technique est suffisamment mature pour être mise en production rapidement, et suffisamment évolutive pour accompagner la croissance de 5 familles pilotes à plusieurs milliers de familles.

L'architecture n'est pas une fin en soi : elle est au service de la confiance. Si les membres font confiance à FamilleConnect, c'est parce qu'ils savent que leurs données sont sécurisées, que l'application fonctionne même en zone rurale, que les paiements Mobile Money sont fiables, et que la mémoire familiale est durablement conservée. Chaque choix technique contribue à cette confiance, et c'est ce qui fera le succès ou l'échec de la plateforme.

Les prochaines étapes consistent à valider cette architecture avec l'équipe technique et les partenaires, à produire les premiers schémas de données détaillés, et à lancer le développement du MVP (Phase 1) avec un périmètre strictement limité aux univers 1 et 4, sur 9 mois.

Principes à retenir

- 1. Offline-first n'est pas optionnel au Cameroun : il structure toute l'architecture front-end et influence les choix back-end.
- 2. La sécurité est un principe structurant, pas une couche ajoutée : chiffrement, 2FA, immutabilité, RBAC dès la conception.
- 3. La scalabilité horizontale permet une croissance des coûts alignée sur les revenus, sans rupture architecturale.
- 4. Le coût par membre doit rester < 500 FCFA/mois, sous peine de rendre le modèle économique non viable.
- 5. L'interopérabilité (Mobile Money, calendriers, push) est un facteur critique d'adoption, à concevoir nativement.

FamilleConnect, par son architecture technique, veut être la plateforme qui tient ses promesses — fiabilité, sécurité, performance, accessibilité — et qui, par cette tenue, gagne la confiance des familles pour longtemps.